

Git, Mercurial, and Subversion are popular version control applications, although Git is by far the most popular. In this chapter, you'll learn how to set up files for code projects and use Git to track their changes.

Git Commits and Repos

Git allows you to save the state of your project files, called *snapshots* or *commits*, as you make changes to them. That way, you can roll back to any previous snapshot if you ever need to. Commit is a noun and a verb; programmers commit (or save) their commits (or snapshots). *Check-in* is also a less popular term for commits.

Version control systems also make it easy for a software developer team to remain in sync with each other while they make changes to a project's source code. As each programmer commits their changes, other programmers can pull these updates onto their computers. The version control system tracks what commits were made, who made them, and when they made them, along with the developers' comments describing the changes.

Version control manages a project's source code in a folder called a repository, or *repo*. In general, you should keep a separate Git repo for each project you're working on. This chapter assumes you're mostly working on your own and don't need the advanced Git features, such as branching and merging, that help programmers collaborate. But even if you're working alone, there is no programming project too small to benefit from version control.

Using Cookiecutter to Create New Python Projects

We call the folder that contains all the source code, documentation, tests, and other files related to a project the *working directory* or *working tree* in Git parlance, and *project folder* more generally. The files in the working directory are collectively called the *working copy*. Before we create our Git repo, let's create the files for a Python project.